MAJOR MILESTONES B.V.

SAFe® Transformaties — Van Strategie naar Executie

Documentchaos in je Agile transformatie? Een slim ontworpen GMS lost het op (Deel 1)

Van privéhuishouden naar organisatie — exact dezelfde principes, alleen grotere schaal.

Maart 2026

ing. R.P. Peels

SAFe® Practice Consultant (SPC) | Authorised SAFe® Trainer

Documentchaos in je Agile transformatie? Een slim ontworpen GMS lost het op (Deel 1)

Van privéhuishouden naar organisatie — exact dezelfde principes, alleen grotere schaal.

ing. R.P. Peels · Maart 2026

1. Inleiding

In kleine organisaties lijkt documentbeheer/informatiemanagement vaak nog overzichtelijk. Maar zodra organisaties groter worden groeit het aantal documenten explosief. Dit geldt in het bijzonder voor Agile transformaties. Een serieuze transformatie met multiple portfolio's, Solution Trains en Agile teams produceren enorme hoeveelheden informatie: rollenbeschrijvingen, policies, Target Operating Model met tientallen hoofdstukken, architectuurdocumenten, werkinstructies, presentaties, assessments, trainingsmateriaal en honderden detailontwerpen. Dit kan gemakkelijk oplopen tot >100.000 documenten binnen 2 jaar tijd. Ik heb meerdere keren bij opdrachtgevers gezeten met >300.000 confluence pages of sharepoint documenten.

Het probleem is niet dat de documenten ontbreken. Het probleem is dat de organisatie ze kwijt is doordat onduidelijk is welke informatie je waar moet opslaan. Medewerkers zoeken naar het laatste Target Operating Model, het laatste LPM-ontwerp, contracten, besluiten, beleidsdocumenten. Ze weten zeker dat die documenten bestaan. Maar waar ze staan, is een ander verhaal. Laat staan wat exact de laatste versie is.

Daarmee ontstaat een fundamentele vraag:

> *Hoe zorg je ervoor dat je agile transformatie in control blijft over haar transformatie informatie?*

2. Leren via herkenbare privévoorbeelden met 2 bouwstenen

Het interessante is dat dit probleem niet alleen in organisaties bestaat. Vrijwel iedereen kent het ook uit zijn privésituatie. Documenten bestaan ergens op je gezin's harde schijf, maar vaak weet niemand precies waar. Je zoekt naar een polis, een factuur, je hypotheek nota, een diploma of een document waarvan je zeker weet dat je het ooit hebt gemaakt maar waar had je die toen opgeslagen?

Dit waarschijnlijk voor iedereen herkenbare probleem vormt het vertrekpunt van dit artikel. Omdat elke organisatie anders is, leg ik de oplossing niet uit aan de hand van een specifieke organisatie, maar aan de hand van een privésituatie die iedereen begrijpt. Iedereen kent namelijk zijn eigen huishouden en zijn eigen documenten. Daardoor zijn voorbeelden direct herkenbaar.

Wanneer je de principes eenmaal begrijpt, kun je ze vervolgens eenvoudig vertalen naar de informatiehuishouding van je eigen organisatie.

De aanpak die ik gebruik bestaat uit twee bouwstenen:

- **GMS – Generieke Mappenstructuur:** De logische structuur waarin documenten worden opgeslagen.
- **BNC – BestandsNaamConventie:** Een standaard manier om bestanden te benoemen zodat documenten ook buiten hun map herkenbaar blijven.

Dit artikel gaat over het **eerste onderdeel:** de **Generieke Mappenstructuur (GMS)**. Het tweede onderdeel — de **BestandsNaamConventie (BNC)** — wordt in een aparte publicatie behandeld. Samen vormen deze twee onderdelen een eenvoudige maar krachtige methode om jouw documentbeheer structureel op orde te brengen.

In de rest van dit artikel laat ik zien hoe je een **Generieke Mappenstructuur (GMS)** ontwerpt waarmee je documenten niet alleen opslaat, maar ook altijd weer kunt terugvinden i.c.m. een BNC.

3. Waarom Full Text Retrieval alleen niet werkt

In discussies over documentbeheer hoor je regelmatig het volgende argument: als zoekmachines tegenwoordig zo goed zijn, waarom zouden we dan nog mappenstructuren en naamgevingsregels nodig hebben? Het idee klinkt aantrekkelijk. Je gooit alle bestanden in één grote bak, of 1 map, en laat een Full Text Retrieval engine alles indexeren. Daarna hoef je alleen nog maar een zoekterm in te typen.

In de praktijk werkt dit echter minder goed. Het eerste probleem is dat je nog steeds moet weten wát je zoekt. Je moet een woord kunnen bedenken dat in het document voorkomt. Als je dat woord niet meer weet, of als het document andere terminologie gebruikt dan jij in gedachten hebt, dan blijft het document onvindbaar. Het tweede probleem is dat niet elk document doorzoekbare tekst bevat. Denk aan gescande PDF's zonder OCR, foto's, afbeeldingen, ZIP-bestanden of oude documenten. Voor een zoekmachine bestaan deze bestanden wel, maar ze zijn inhoudelijk niet te indexeren. Ze verdwijnen daardoor stilletjes in de vergaarbak.

> *Daarom zie ik Full Text Retrieval niet als oplossing, maar als hulpmiddel i.c.m. met mijn GMS en BNC aanpak.*

4. Vinger oefening

Voordat we de ontwerpprincipes van een goede GMS behandelen, is het nuttig om eerst een eenvoudige oefening te doen. Zie onder ongeveer dertig documenten die vrijwel ieder huishouden kent. Probeer daarvoor zelf nu eens logische mappenstructuur voor te bedenken. Het doel van deze oefening is niet om direct de perfecte indeling te maken, maar om te ervaren hoe snel ogenschijnlijk eenvoudige documenten al classificatievragen oproepen. Kijk bij elk document in welke soort map je het zou opslaan, volgens welk ordeningsmechanisme ga je ordenen?

Document	Document	Document	Document
Paspoort	Rijbewijs	Geboorteakte	Huwelijksakte / samenlevingscontract
DigiD-gegevens / BSN-overzicht	Koopakte / huurcontract woning	Hypotheekakte	Eigendomsbewijs (kadaster)
Verzekeringspolis opstal	Verzekeringspolis inboedel	Energiecontract (stroom/gas)	Servicecontract cv-ketel
Jaarlijkse belastingaangifte (IB)	WOZ-beschikking	Bankafschriften (jaaroverzicht)	Salarisstroken
Jaaropgave werkgever	Pensioenopgave	Kentekenbewijs auto	Verzekeringspolis auto (groene kaart)
APK-keuringsrapport	Zorgverzekeringspolis	Vaccinatiebewijs / medisch dossier uittreksel	Tandartsdossier (behandeloverzicht)
Geboorteakte kind	Schooldiploma's / rapporten	Kinderbijslag-beschikking	Reisverzekeringspolis

Document	Document	Document	Document
Levensverzekering / uitvaartverzekering	Testament / notariële volmacht	**Verzekeringspolis Volvo V70**	

Waar ga je in je mappen ontwerp bijvoorbeeld de **verzekeringspolis van je Volvo V70** opslaan? Dat document past namelijk in meerdere categorieën, maar mag uiteindelijk maar **op één logische plek bestaan, lees 1 in 1 map**. Je zou deze polis kunnen opslaan:

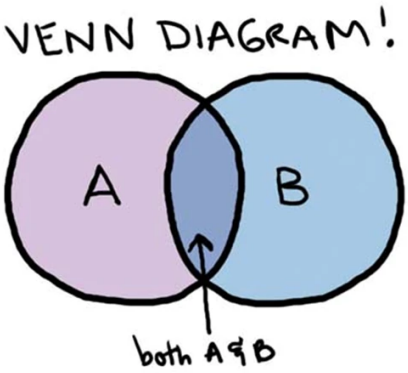
- in de map **Polissen**, samen met al je andere verzekeringen.
- in de map **Volvo V70**, samen met alle autodocumenten van die auto
- in een map **Achmea**, zodat je per leverancier een volledig overzicht hebt wat je daar hebt afgesloten.
- Maak je die map dan aan op D:\\Volvo V70, of is de Volvo een submap van een andere map. Welke en waarom, hoe ben je tot die ordening gekomen?

Precies dit soort twijfelgevallen maakt het ontwerpen van een goede mappenstructuur zeer lastig en snel complex. Privé is dat al ingewikkeld genoeg. In agile transformaties wordt het nog veel complexer, omdat niet alleen het volume toeneemt, maar ook de context, de afhankelijkheden en het aantal mogelijke invalshoeken (document kenmerken).

5. Twee denkmodellen achter een goede structuur

5.1. Venn-diagrammen — waarom mappenstructuren zo lastig zijn

Een Venn-diagram is een manier om verzamelingen en hun overlap visueel te tonen. Elke cirkel staat voor een categorie; de overlap tussen cirkels toont elementen die in meerdere categorieën passen.



Venn-diagram: overlap tussen categorieën

Stel je drie cirkels voor als mappen:

Map A: Alle verzekeringspolissen in één map. Dit is 100% zuiver

Map B: Alle documenten over je Volvo V70. Dit is 100% zuiver

Map C: Wat is Map C?

Nu ontstaat het interessante probleem. Neem bijvoorbeeld de autoverzekeringspolis van je Volvo V70. Dat document hoort tegelijk bij drie categorieën:

- het heeft een verzekeringspolis
- het gaat over facturen

- het gaat specifiek over je Volvo V70

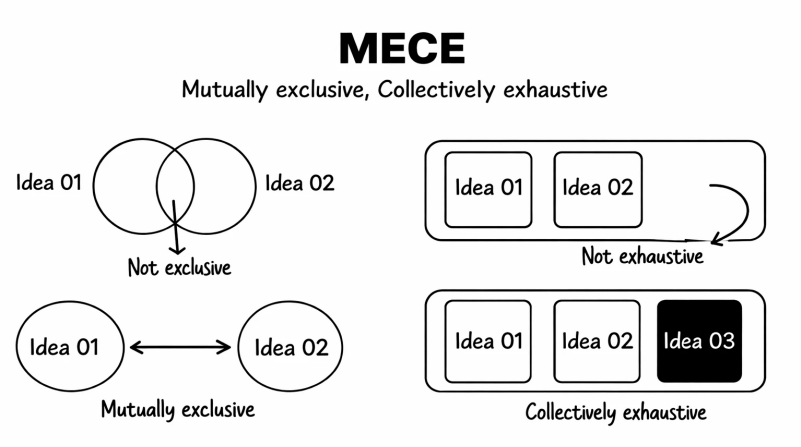
In een Venn-diagram ligt dat document dus precies in de overlap van alle drie de cirkels. Maar in een digitale mappenstructuur kan een bestand maar op één plek staan. Je kunt het document niet tegelijk opslaan in drie verschillende mappen zonder duplicaten of links te maken.

En hier ontstaat het kernprobleem van mappenstructuren: **documenten hebben meerdere kenmerken, maar mappen zijn enkelvoudig**. Zodra je categorieën gebruikt die elkaar overlappen, weet je nooit meer zeker waar een document thuishoort. De één zoekt het document onder "verzekeringen", de ander onder "auto", en weer iemand anders onder "Volvo". Dat is 1 van de kernproblemen van mappenstructuren.

De conclusie is daarom eenvoudig maar belangrijk: overlappende categorieën werken niet als hoofdstructuur. Je moet één ordeningsprincipe kiezen en dat consequent toepassen.

5.2. Het McKinsey MECE-principe

Het Venn-diagram laat zien waarom mappenstructuren zo lastig zijn. Documenten hebben meerdere kenmerken tegelijk. Een autoverzekeringspolis is bijvoorbeeld tegelijk een verzekering, een autodocument en een document over een specifieke auto. Maar een bestand kan maar op één plek staan hadden we al gezegd. Daarom moet je een structuur ontwerpen waarbij categorieën elkaar niet overlappen. Daarvoor gebruiken we het MECE-principe.



MECE: Mutually Exclusive, Collectively Exhaustive

MECE staat voor:

- **Mutually Exclusive** – categorieën mogen elkaar niet overlappen. Dus een document staat in Map A *of* in Map B.
- **Collectively Exhaustive** – samen moeten ze alles afdekken, wat betekent dat we een GMS moeten ontwerpen waar we *al onze (gezins) documenten* in kwijt kunnen.

Met andere woorden: elk document past in precies één categorie, en geen enkel document valt buiten de structuur. De relatie met het Venn-diagram is eenvoudig. Het Venn-diagram laat zien dat documenten meerdere eigenschappen hebben. Het MECE-principe dwingt je vervolgens om één van die eigenschappen te kiezen als hoofdstructuur.

Je kiest dus niet tegelijk voor:

- documenttype (verzekeringspolis, financien)
- object (Volvo V70)

Je kiest één ordeningsprincipe als basis voor je mappenstructuur. De andere eigenschappen kun je vervolgens oplossen met submappen of met een goede bestandsnaam. Daarmee wordt de structuur voorspelbaar. Iedereen weet waar een document hoort, ook zonder te zoeken.

Korte denk oefening: Stel dat je een aantal documenten hebt met deze content: mannen, vrouwen, kinderen, transgenders, hermafrodieten en neutraal. Je moet hiervoor een mappenstructuur ontwerpen waarin elk document eenduidig kan worden opgeslagen.

Je besluit twee mappen te maken om deze documenten op je schijf te archiveren:

- c:\\mannen
- c:\\vrouwen

Dan ontstaat direct het probleem. Waar sla je het document "kind" op? En waar hoort een document met het content "hermafrodiet" thuis?

Een korte MECE-check:

- **Man / Vrouw mappen:** Een man is geen vrouw en een vrouw is geen man. Samen vormen ze: alle volwassenen naar geslacht. ✔ MECE
- **Kind / Volwassene:** Een kind is geen volwassene en een volwassene is geen kind. Een volwassene kan een man of vrouw zijn. Samen vormen ze: alle mensen naar leeftijdscategorie. ✔ MECE
- **Man / Vrouw met extra categorieën:** Wanneer je mappen toevoegt zoals een map voor *kind*, *transgender* of *hermafrodiet*, ontstaat overlap. ✘ **Niet MECE**

Deze eenvoudige oefening laat precies zien waarom een structuur die niet MECE is direct tot onvindbaarheid leidt. Sommige documenten passen niet duidelijk in één map, terwijl andere documenten in meerdere mappen tegelijk zouden kunnen horen.

6. Toe te passen GMS Ontwerp principes

6.1. Het levensarchief

LifeTree Architecture is ontworpen als een levenslang persoonlijk archief. Het doel is om het volledige privéleven van een gezin systematisch te bewaren zoals al je woningen, al voertuigen (gereden en huidig), hypotheken, belastingaangiften, vermogensbeheer, diploma's, medische dossiers, individueel maar ook gezinsfoto's en -video's, aankopen en bijvoorbeeld vakantie informatie. De structuur vormt daarmee een digitale representatie van het gezin privéleven.

6.2. Scheid actieve documenten van historisch archief

In plaats van oude informatie te verwijderen, wordt geschiedenis expliciet bewaard voor bijvoorbeeld de belastingdienst, AIVD screening doeleinden maar ook ingeval van overlijden.

Daarom gebruiken we patronen zoals:

- Huidige woning / Historische woningen
- Huidige werkgever / Historische werkgevers
- Huidige school / Historische scholen

Hierdoor ontstaat een volledig historisch levenspad. Ingeval van schijfruimte te kort, of geen interesse meer, kun je gewoon een paar historische mappen deleten.

6.3. Innovatie binnen je mappenstructuur

Een robuuste mappenstructuur ontstaat zelden perfect in één keer. Vooral in het begin (from scratch) zul je regelmatig moeten bijschaven, mappen verplaatsen of een structuur iets aanpassen. Dat is normaal. De eerste periode gebruik je de structuur intensief en ontdek je waar iets beter kan. Check continue hoe MECE je ontwerp is.

Na verloop van tijd stabiliseert je structuur. Uit mijn eigen ervaring blijkt dat wanneer het hoogste niveau en de eerste twee of drie niveaus goed zijn ontworpen, deze daarna vaak tientallen jaren vrijwel ongewijzigd blijven.

6.4. Zoek het hoogste aggregatieniveau

Bij het ontwerpen van een mappenstructuur moet je altijd eerst zoeken naar het hoogste logische aggregatieniveau. Op dat niveau probeer je de hoofdmappen zo MECE mogelijk te maken: Mutually Exclusive en Collectively Exhaustive. Pas daarna werk je verder naar lagere niveaus (lees: submappen) in de boom.

Veel mensen maken de fout om direct mappen aan te maken voor concrete dingen zoals "d:\\Auto" of "d:\\Hypotheek". Dat lijkt logisch, maar leidt bijna altijd en met name snel tot *chaos*. Binnen korte tijd ontstaan er overlappende mappen, dubbele documenten en onduidelijkheid over waar iets hoort. Daarom is de informatiehuishouding bij veel bedrijven zo'n bende.

Begin je GMS ontwerp altijd met een inventarisatie van je documenten en zoek naar een hoger abstractieniveau waar deze onder vallen. Neem bijvoorbeeld een auto. Je zou een map "Auto" kunnen maken. Maar een auto is eigenlijk onderdeel van een grotere categorie: voertuigen. En voertuigen zijn weer onderdeel van een nog hogere categorie: bezittingen of assets.

De vraag is dus steeds: wat is het hoogste logische niveau waar dit onder valt?

Voor een gemiddeld huishouden zou een eenvoudig en logisch hoogste niveau er bijvoorbeeld zo uit kunnen zien:

- Foto's
- Video's
- Geluid
- Documenten
- Assets

Wanneer dit hoogste niveau goed is gekozen en MECE is, kun je daarna pas beginnen met het verder uitwerken van de onderliggende mappenstructuur.

6.5. Perfecte MECE mappenstructuur bestaat niet

Hoewel je een GMS zoveel mogelijk MECE ontwerpt, is 100% MECE in de praktijk onhaalbaar. De reden is fundamenteel: documenten hebben (bijna) altijd meerdere kenmerken tegelijk, maar een mappenstructuur kent slechts één hiërarchie. Je kunt een bestand nu eenmaal maar op één plek opslaan.

Neem de factuur van je Volvo V70. Dat document is tegelijk een factuur, asset document en een document over een specifiek voertuig. Alle drie de categorieën zijn correct. Toch moet je kiezen: gaat het in de map 'Volvo V70', in 'Aankoop facturen 2026' of ergens anders? Er is geen objectief juist antwoord. Wat telt is dat je een keuze maakt en die consequent volhoudt.

Dit brengt ons bij een belangrijk principe dat hier direct uit volgt:

> *Kies één ordeningsprincipe en pas het consequent toe en onderwijs de mapgebruikers hierin.*

Dus niet per map een andere logica, maar overal dezelfde redenering en dezelfde ordeningsprincipes.

6.6. Herhaalbare patronen maken je GMS goed schaalbaar

Veel onderdelen van het leven volgen dezelfde schaallogica. Denk aan gezinsleden, werkgevers, voertuigen, opleidingen, belastingjaren of woningen. Deze categorieën hebben vaak een vergelijkbare interne structuur. Daarom werken we in een Generieke Mappenstructuur met **herhaalbare mappatronen**. Wanneer een nieuw element ontstaat, hoef je de structuur niet opnieuw te ontwerpen. Je kopieert simpelweg een bestaand patroon en past de naam aan.

Voorbeelden:

- **Nieuw gezinslid:** Komt er een nieuw kind in het gezin, dan kopieer je de bestaande mappenstructuur van een ander gezinslid en hernoem je deze naar de naam van het nieuwe kind.
- **Nieuw belastingjaar:** Wanneer een nieuw fiscaal jaar begint, kopieer je bijvoorbeeld de map *Aangifte IB 2024* naar *Aangifte IB 2025*. De onderliggende structuur blijft intact en kan direct opnieuw gebruikt worden.
- **Nieuwe werkgever of opleiding:** Ook hier kun je een bestaand patroon kopiëren zodat contracten, correspondentie, beoordelingen en relevante documenten direct logisch geordend zijn.

Het grote voordeel hiervan is dat het systeem **meegroeit met je leven** zonder dat je telkens opnieuw hoeft na te denken over de structuur. En, je hoofd structuur, de rootmappen blijven onaangetast.

6.7. Beperk padlengte door mapnamen niet te lang te maken

Windows kent een maximale lengte voor een volledig bestandspad. Zo'n pad bestaat uit de volledige route naar een bestand: alle mapnamen + de bestandsnaam zelf.

In oudere Windows-versies en in veel software geldt nog steeds de klassieke MAX_PATH-limiet van 260 tekens. Moderne versies van Windows 10 en Windows 11 kunnen technisch langere paden aan (tot ongeveer 32.767 tekens), maar veel programma's, backupsoftware en cloudopslag ondersteunen dit niet altijd volledig. In de praktijk is het daarom verstandig te ontwerpen alsof de 260-tekenslimiet nog steeds bestaat.

Daarom gebruikt een goed GMS ontwerp korte en compacte mapnamen, zoals *Diplomas*, *Medisch*, *Werk*, *IB 2025*, *Autos* (geen Voertuigen, dat zijn meer chars).

Door namen kort te houden blijft er ruimte over voor extra submappen en met name BNC compliant bestandsnamen (die zijn nm langer dan gemiddeld), zonder dat het systeem tegen technische grenzen aanloopt en je windows errors genereert.

6.8. Nummer je mappen om de juiste volgorde af te dwingen

Besturingssystemen sorteren mappen standaard alfabetisch. Dat betekent dat de volgorde niet altijd overeenkomt met de logica die jij in je hoofd hebt. Door mappen te nummeren kun je de volgorde zelf bepalen. Daarom is het verstandig om een vast formaat te gebruiken, bijvoorbeeld:

- ###) mapnaam1
- ###) mapnaam2

In zowel privé- als organisaties situaties zijn drie cijfers ruim voldoende, omdat je zelden meer dan 999 submappen onder de hoger liggende-map nodig hebt. Het voordeel daarvan is dat je later nog makkelijk een map ertussen kunt plaatsen, bijvoorbeeld 015) of 025), zonder dat je alle mappen opnieuw moet nummeren. Daardoor blijft de mappenboom overzichtelijk en logisch opgebouwd.

Tip: ingeval van vakanties mappen, werkgevers, etc is het handig om YYMMDD (YYYYMMDD als je al vrij lang op deze aarde rondloopt) toe te passen of het jaartal waar de map over gaat, zoals aangifte IB. In alle gevallen staan de mappen dan automatisch op volgorde. Zie toepassing in de voorbeeld GMS.

6.9. Map maken of wens oplossen in de bestandsnaam?

Bij het ontwerpen van een mappenstructuur ontstaat regelmatig de vraag: *"moet ik hiervoor een nieuwe map maken, of kan ik dit beter oplossen in de bestandsnaam?"*

Veel mensen maken automatisch een nieuwe map per document attribuut. Dat lijkt logisch, maar elke extra map maakt de structuur complexer. Na verloop van tijd ontstaat dan een steeds diepere boom met overlappende categorieën en verliest men het overzicht. Daarom is het verstandig eerst te bepalen of iets een **echte categorie** is, of slechts een **attribuut van een document**.

Neem het voorbeeld van **abonnementen**. Denk aan Vodafone, Netflix of een sportschool. Je zou hiervoor een map kunnen maken:

```
020) Financiën
└── 090) Abonnementen
```

Maar een abonnement is beter geen aparte documentcategorie. Het is meestal een **contract of financieel document (de factuur) met het kenmerk "abonnement"**. In dat geval kun je het document ook gewoon opslaan binnen je financiële administratie en het attribuut in de bestandsnaam verwerken.

Bijvoorbeeld:

```
240315 contract mobielabonnement -Vodafone.pdf
240201 factuur Abonnement streaming -Netflix.pdf
230901 polis autoverzekering VolvoV70 -Achmea.pdf
```

Met zoektools zoals **Windows Search (F3)** of **Everything** kun je deze documenten vervolgens eenvoudig terugvinden door te zoeken op bijvoorbeeld:

```
*abonnement*.pdf
```

Het principe is dus eenvoudig:

- **Gebruik mappen voor stabiele categorieën.**
- **Gebruik bestandsnamen voor documentkenmerken (attributen).**

Hoe je zulke bestandsnamen systematisch opbouwt, bespreek ik in **deel 2: de BestandsNaamConventie (BNC)**. Samen met de GMS vormt dit de tweede bouwsteen om documenten altijd snel terug te vinden.

6.10. Mapnamen zijn zelf verklarend en tijdloos

Een mapnaam is een belofte aan de toekomstige gebruiker over wat erin zit. Die gebruiker ben jij zelf, over tien jaar, of een gezinslid, of agile transformatie lid dat de structuur niet heeft ontworpen.

Vermijd daarom altijd namen als *Nieuw*, *Oud*, *Diversen*, *Overig*, *Tijdelijk* of *Misc*. Deze namen zeggen niets over de inhoud en groeien zonder uitzondering uit tot een vergaarbak. En dat willen we niet. Alles wat nergens anders past, belandt er vroeg of laat in. Daarmee ondermijnen ze precies wat je met een GMS wil bereiken.

Een goede mapnaam is kort ivm max padlengte, concreet, contextonafhankelijk en moet begrijpelijk zijn zonder dat je de rest van de structuur hoeft te kennen. Denk aan: Autos, Diplomas, Belasting, Investments, Werkgevers, Medisch: iedereen begrijpt direct wat erin zit, nu en over tien jaar.

6.11. Wijs de GMS Admin aan

Een GMS werkt alleen als de structuur stabiel blijft. Zodra meerdere mensen zonder spelregels mappen mogen aanmaken, groeit de structuur ongecontroleerd. Binnen korte tijd ontstaan parallelle mappen voor hetzelfde doel en verliest de GMS zijn voorspelbaarheid. De ontwerper weet vaak alles van de opbouw en wat waar en is getrained in MECE denken en doen, Venn diagrammen.

Daarom geldt een eenvoudig principe: er is altijd één hoofd admin (binnen de familie) die de hoofdstructuur beheert en bewaakt. In een transformatie is dat bijvoorbeeld iemand uit de LACE, VMO of iemand van een Enabler team.

Een absolute spelregel is dat rootstructuur, tot zeg map niveau 3, nooit zonder toestemming worden aangepast, die vormt de ruggengraat van het systeem. Lagere niveaus zouden (liever niet!) vrijer worden uitgebreid, zolang ze het vastgestelde herhaalbare patroon volgen.

In agile transformaties is dit principe extra relevant. Teams maken al snel hun eigen mappenstructuren die logisch voelen vanuit hun eigen context, maar niet aansluiten op de bredere GMS. Daarom, één admin die de spelregels bewaakt voorkomt dit.

7. Kers op de taart: Een uitgewerkte ~MECE privé GMS

Nu we alle ontwerp principes hebben behandeld, is het tijd voor de kers op de taart: een volledig uitgewerkte ~MECE privé GMS die je direct kunt gebruiken. Hieronder lopen we elk ontwerp principe één voor één langs en toetsen we of de GMS hieraan voldoet — en waarom we soms bewust afwijken.

7.1. Principe: Het levensarchief

■ Compliant. De GMS is ontworpen als een levenslang persoonlijk archief. De rootmap D:\\ dekt het volledige privéleven van een gezin, als uitgangspunt van mijn voorbeeld gezin (!): personen (Vader, Moeder, kinderen), huisdieren, gezinsfinanciën, media en assets. Er is geen einddatum, geen projectmap die vervalt, geen tijdelijke structuur. De structuur is ingericht om tientallen jaren mee te gaan en het volledige gezinsleven te bevatten, inclusief historische lagen.

7.2. Principe: Scheid actieve documenten van historisch archief

■ Compliant. Dit principe is consequent doorgevoerd. Bij werk geldt het patroon '010) Huidige werkgever / 020) Historische werkgevers'. Bij school: '010) Huidige school / 020) Historische scholen'. Bij woning: '010) Huidige woning / Historische woningen Woning1, Woning2, Woning3'. Actief en historisch zijn zo altijd gescheiden, maar wel samenhangend bewaard. Oude informatie verdwijnt niet — ze krijgt een vaste historische plek.

7.3. Principe: Innovatie binnen je mappenstructuur

■ Compliant. De GMS bevat expliciet een map '070) Gezins Innovaties' onder Administratie, met submappen Onderzoek, Offertes, Vergelijkingen en Besluiten. Dit is de plek waar een gezin nieuwe ideeën, aanschaffingen of verbeteringen doordenkt voordat ze een besluit nemen. Zo is innovatie — het continu bijstellen van je eigen informatiehuishouding — structureel verankerd in de GMS zelf.

7.4. Principe: Zoek het hoogste aggregatieniveau

■ Compliant. De vier rootmappen van de GMS zijn: 010) Administratie, 020) Gezins Financiën, 030) Media en 040) Assets. Dit zijn de hoogst mogelijke abstractieniveaus voor een huishouden. Er staat geen map 'Auto', 'Hypotheek' of 'Achmea' op rootniveau — die zitten dieper in de boom, op de juiste logische plek. Door eerst het hoogste niveau te bepalen en pas daarna verder te verfijnen, blijft de hoofdstructuur stabiel en voorspelbaar.

7.5. Principe: Perfecte MECE mappenstructuur bestaat niet

■ Bewust ~MECE. De naam van deze GMS heet niet voor niets '~MECE'. Er zijn grensvlakken in de structuur waar een document in meerdere mappen zou kunnen passen. Neem de verzekeringspolis van een auto: die past zowel onder 040) Assets > Roerend Goed > Voertuigen > Auto Vader als onder een eventuele polissenmap. De keuze in deze GMS is om alles rondom een specifiek object bij dat object te bewaren. Dat is een bewuste, consistente keuze — geen fout. Het principe geldt: kies één ordeningsprincipe en pas het consequent toe.

7.6. Principe: Herhaalbare patronen maken je GMS goed schaalbaar

■ Compliant. De GMS werkt uitgebreid met herhaalbare patronen. Elk gezinslid (Vader, Moeder, Kind1, Kind2) heeft exact dezelfde interne mapstructuur: Identiteit, Diplomas, Medisch, Werk/School, Contracten. Elk historisch voertuig volgt hetzelfde patroon (Auto1, Auto2, Auto3). Belastingaangiften per jaar worden herhaald (2022, 2023, 2024, 2025). Vakantiefotos per jaar idem. Wanneer er een Kind3 bijkomt, kopieer je simpelweg de structuur van Kind1 of Kind2 en pas je de naam aan. De GMS groeit mee met het leven zonder herontwerp.

7.7. Principe: Beperk padlengte door mapnamen niet te lang te maken

■ Compliant. Alle mapnamen in de GMS zijn kort en compact gehouden. Voorbeelden: Identiteit, Diplomas, Medisch, Werk, Contracten, Autos, Witgoed, Bruingoed, Grijsgoed, Grond. Langere alternatieven zoals 'Voertuigen en Transportmiddelen' of 'Historische Werkgevers Archief' zijn vermeden. Ook de rootnamen zijn compact: Administratie (14 tekens), Media (5 tekens), Assets (6 tekens). Dit houdt het volledige pad ruim binnen de praktische Windows-limiet van 260 tekens, ook bij diepere submappen en lange BNC-bestandsnamen.

7.8. Principe: Nummer je mappen om de juiste volgorde af te dwingen

■ Compliant. De GMS gebruikt consequent het formaat '##) Mapnaam' op elk niveau. Op rootniveau: 010) Administratie, 020) Gezins Financiën, 030) Media, 040) Assets. Binnen Administratie: 010) Vader, 020) Moeder, 030) Kind1, 040) Kind2, 050) Hond enzovoort. Binnen een persoon: 010) Identiteit, 020) Diplomas, 030) Medisch, 040) Werk, 050) Contracten. Dit dwingt de logische volgorde af — de meest essentiële en meest geraadpleegde mappen staan bovenaan, onafhankelijk van alfabetische sortering door het besturingssysteem.

7.9. Principe: Mapnamen zijn zelfverklarend en tijdloos

■ Compliant. Geen enkele mapnaam in de GMS heet 'Diversen', 'Overig', 'Tijdelijk', 'Nieuw' of 'Misc'. Alle namen zijn concreet en contextonafhankelijk: Identiteit, Diplomas, Medisch, Vaccinaties, Chip registratie, Witgoed, Bruingoed, Grondstoffen, Vastgoed investeringen, Jaaroverzichten broker. Een willekeurige gebruiker die deze structuur voor het eerst ziet, begrijpt direct wat er in elke map te vinden is — zonder uitleg en ook tien jaar later nog.

7.10. Principe: Wijs de GMS Admin aan

■ Compliant. De GMS is ontworpen vanuit één ordeningsprincipe, met een stabiele rootstructuur die niet zomaar mag worden aangepast. In een gezinscontext is dit de 'GMS Admin' — doorgaans degene die de structuur heeft ontworpen en begrijpt. De rootmappen (niveau 1 t/m 3) vormen de ruggengraat en worden niet zonder overleg

gewijzigd. Lagere niveaus mogen beperkt worden uitgebreid, zolang het vastgestelde herhaalbare patroon wordt gevolgd. Voor wie deze GMS als startpunt gebruikt: wijs direct bij ingebruikname één persoon aan als beheerder.

De uitgewerkte GMS in Bijlage 1 voldoet dus aan alle tien ontwerp principes. Waar we bewust afwijken — zoals bij de ~MECE keuze om alles rondom een object bij dat object te bewaren — is dit een doordachte en consistente beslissing, geen vergissing. De structuur is vrij te gebruiken als startpunt voor je eigen privé informatiehuishouding. Pas de namen van gezinsleden, voertuigen en huisdieren aan naar je eigen situatie, en de GMS is direct inzetbaar.

8. Tot slot: Tips voor een robuuste GMS

8.1. Scheidt data van applicaties, zorg voor goede backups

Het is raadzaam om je programma- en systeemschijf (C:\\) te scheiden van de schijf waarop je persoonlijke gegevens staan, vaak D:\\. De reden hiervoor is eenvoudig. De C-schijf kan worden vervangen, mag crashen. Je programma's en besturingssysteem zijn vervangbaar. Je gegevens zijn dat niet. Je documenten, foto's, contracten en administratie moeten daarom op een aparte schijf staan die je zo veilig mogelijk beheert. Idealiter staat deze gegevensschijf op een redundante opslag, bijvoorbeeld een RAID-opstelling (zoals RAID-5 of RAID-1), zodat een defecte schijf niet direct dataverlies betekent.

Daarnaast hoort bij belangrijke gegevens altijd een twee-ledige backupstrategie:

- een frequente online backup (bijvoorbeeld cloud of NAS-backup)
- een periodieke offline backup op een externe schijf die niet permanent met internet verbonden is (i.v.m. Crypto locking)

Die offline backup bewaar je bij voorkeur op een andere fysieke locatie of op een veilige plek in huis.

8.2. Automatisch de hele Lifetree op je PC zetten?

Wil je de volledige Lifetree GMS direct op je eigen computer hebben staan, zonder zelf alle mappen handmatig aan te moeten maken? Stuur me dan even een berichtje. Ik heb namelijk een klein programmaatje ontwikkeld dat de complete mappenstructuur automatisch aanmaakt op je desktop. Zo kun je direct aan de slag en de kracht van een goed ontworpen GMS in je eigen praktijk toetsen, zonder enige voorkennis of technische handelingen.

9. Samenvatting

Na het lezen van dit artikel weet je het volgende.

Documentchaos is geen toeval. In zowel privésituaties als grote Agile transformaties ontstaat informatiechaos vrijwel altijd door hetzelfde grondprobleem: documenten hebben meerdere kenmerken tegelijk, maar een mappenstructuur kent slechts één hiërarchie. Wie dit niet onderkent, bouwt onbewust een systeem dat zichzelf ondermijnt.

De oplossing bestaat uit twee bouwstenen: een Generieke Mappenstructuur (GMS) en een BestandsNaamConventie (BNC). Dit artikel behandelt de GMS. De BNC komt aan bod in Deel 2.

Een goede GMS ontwerp je op basis van tien principes: je kiest het hoogste aggregatieniveau als vertrekpunt, past het MECE-principe zo consequent mogelijk toe (Mutually Exclusive, Collectively Exhaustive), scheidt actieve documenten van historisch archief, werkt met herhaalbare patronen voor schaalbaarheid, nummert mappen voor de juiste volgorde, houdt mapnamen kort en tijdloos, lost documentattributen op in de bestandsnaam in plaats van extra mappen te maken, bewaakt de structuur via één aangewezen GMS Admin, en accepteert dat 100% MECE in

de praktijk onhaalbaar is — zolang je één ordeningsprincipe consequent volhoudt.

Dezelfde principes die thuis werken, werken ook in een Agile transformatie — net zoals in elke andere organisatie. De schaal verschilt, de logica niet. Een privé GMS met vier rootmappen dekt een heel gezinsleven. Een organisatie GMS met vergelijkbare principes dekt honderden duizenden documenten over meerdere portfolios, Solution Trains en teams.

Morgen direct beginnen? De volledig uitgewerkte ~MECE privé GMS staat in Bijlage 1 van dit artikel, klaar om als startpunt te gebruiken. Pas de namen van gezinsleden, voertuigen en huisdieren aan naar je eigen situatie — en je GMS is direct inzetbaar. Wil je de mappenstructuur automatisch laten aanmaken op je eigen pc, stuur me dan even een berichtje (zie sectie 8.2).

10. Bijlage: Een uitgewerkte ~MECE privé GMS

Onderstaand overzicht toont een voorbeeld van een **Generieke Mappenstructuur** voor een huishouden. De structuur is logisch opgebouwd, MECE op hoofdniveau en bedoeld om tientallen jaren bruikbaar te blijven.

10.1. Hoofdniveau

Dit is het **hoogste aggregatieniveau** van de Generieke Mappenstructuur. Op dit niveau zijn de categorieën ontworpen volgens het MECE-principe (Mutually Exclusive, Collectively Exhaustive).

```
C:\
├── 010) Administratie
├── 020) Financiën
├── 030) Media
└── 040) Assets
```

10.2. Detailniveau

```
D:\
|
├── 010) Administratie
| ├── 010) Vader
| | ├── 010) Identiteit
| | ├── 020) Diplomas
| | ├── 030) Medisch
| | ├── 040) Werk
| | | ├── 010) Huidige werkgever
| | | └── 020) Historische werkgevers
| | | ├── 980315) Werkgever1
| | | ├── 051101) Werkgever2
| | | └── 140601) Werkgever3
| | ├── 050) Contracten
| | ├── 060) Verzekeringen
| | ├── 070) Pensioen
| | └── 080) Bankzaken
| |
| ├── 020) Moeder
| | ├── 010) Identiteit
| | ├── 020) Diplomas
| | ├── 030) Medisch
| | ├── 040) Werk
```

```
| | | ├── 010) Huidige werkgever
| | | └── 020) Historische werkgevers
| | | ├── 990901) Werkgever1
| | | ├── 070301) Werkgever2
| | | └── 150901) Werkgever3
| | ├── 050) Contracten
| | ├── 060) Verzekeringen
| | ├── 070) Pensioen
| | └── 080) Bankzaken
| |
| ├── 030) Kind1
| | ├── 010) Identiteit
| | ├── 020) Diplomas
| | ├── 030) Medisch
| | ├── 040) School
| | | ├── 010) Huidige school
| | | ├── 020) Historische scholen
| | | | ├── 840901) Lagere school
| | | | ├── 900901) Middelbare school
| | | | └── 960901) Hogere school
| | | └── 030) Cursussen
| | ├── 050) Overig
| | └── 060) Financieel
| |
| ├── 040) Kind2
| | ├── 010) Identiteit
| | ├── 020) Diplomas
| | ├── 030) Medisch
| | ├── 040) School
| | | ├── 010) Huidige school
| | | ├── 020) Historische scholen
| | | | ├── 840901) Lagere school
| | | | ├── 900901) Middelbare school
| | | | └── 960901) Hogere school
| | | └── 030) Cursussen
| | ├── 050) Overig
| | └── 060) Financieel
| |
| └── 050) Huisdier 1
| ├── 010) Dierenarts
| ├── 020) Vaccinaties
| ├── 030) Verzekering
| ├── 040) Chip registratie
| └── 050) Overig
|
├── 020) Financiën
| ├── 010) Aangifte IB
| | ├── 010) Vader
| | | ├── 2024
| | | └── 2025
| | └── 020) Moeder
| | ├── 2024
| | └── 2025
| ├── 020) Gezins Facturen
| | ├── 2024
| | └── 2025
```

```
│ ├── 040) Toeslagen
│ │ ├── 010) Zorgtoeslag
│ │ ├── 020) Kinderbijslag
│ │ └── 030) Huurtoeslag
│ ├── 060) Investments
│ │ ├── 010) ETF
│ │ ├── 020) Crowdfunding
│ │ ├── 030) Grondstoffen
│ │ ├── 040) Cryptos
│ │ └── 050) Vastgoed investeringen
│ ├── 080) Notarieel
│ │ ├── 010) Testament
│ │ ├── 020) Volmachten
│ │ ├── 030) Huwelijk of samenleving
│ │ └── 040) Erfrecht
│ └── 090) Gezins Abonnementen (kan je mergen met facturen)
│ ├── 2024
│ └── 2025
│
├── 030) Media
│ ├── 010) Fotos
│ │ ├── 010) Gezin
│ │ ├── 020) Vakanties
│ │ │ ├── 210403 Z
│ │ │ ├── 890504 X
│ │ │ └── 961023 Y
│ │ ├── 030) Vader
│ │ ├── 040) Moeder
│ │ ├── 050) Kind1
│ │ ├── 060) Kind2
│ │ └── 070) Huisdier 1
│ ├── 020) Videos
│ │ └── 010) tbd
│ └── 030) Geluid
│ └── 010) tbd
│
└── 040) Assets
├── 010) Software
│ ├── 010) Installatie kits
│ └── 020) Licenties
└── 020) Fysieke bezittingen
├── 010) Roerend Goed
│ ├── 010) Voertuigen
│ │ ├── 050301) Auto vader
│ │ │ └── 010) Historische auto's
│ │ │ └── 990410) Auto1
│ │ ├── 120615) Auto moeder
│ │ │ └── 010) Historische auto's
│ │ │ └── 120615) Auto1
│ │ ├── 150901) Aanhanger
│ │ └── 190315) Elektrische fiets
│ ├── 020) Witgoed
│ │ └── 010) Koelkast
│ ├── 030) Bruingoed
│ │ ├── 010) Televisie
│ │ └── 020) Platenspeler
```

```
│ └── 040) Grijsgoed
│ ├── 010) Laptop
│ ├── 020) Netwerk
│ └── 030) Randapparatuur
└── 020) Onroerend Goed
├── 010) Huidige woning
│ └── 010) Historie
│ ├── 030614) Woning2
│ └── 181205) Woning3
├── 020) Vakantieverblijf
└── 030) Grond
```

– einde –